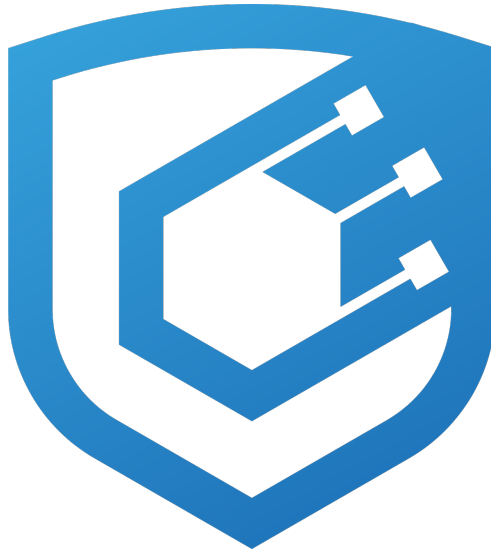




Algo Stablecoin Protocol

Security Audit Report

Version 1.0

Prepared by: Farhan Davin (Lead Auditor)

September 28, 2026

Algo Stablecoin Protocol Security Audit Report

Farhan Davin

September 28, 2026

Prepared by: Farhan Davin (Lead Security Researcher)

Audited Codebase: CodeHawks-Contests / ai-algo-ssstablecoinsss

Target Blockchain: ZKsync Era (Vyper)

Methodology: The Tincho Method

1. Executive Summary

Algo Ssstablecoinsss is a decentralized, exogenous collateral-backed stablecoin protocol designed for deployment on ZKsync Era. The system enables users to deposit collateral assets (specifically **WETH** and **WBTC**) to mint a USD-pegged decentralized stablecoin (**DSC**). The protocol maintains stability through an algorithmic overcollateralization model (minimum Health Factor of 1.0) with an external liquidator incentive mechanism (10% liquidation bonus).

A comprehensive security review was performed using **The Tincho Method**, emphasizing empirical proof of vulnerabilities over theoretical conjecture. Every identified vulnerability is backed by executable tests, mathematical proofs, and automated mutation scorecards.

Key Finding Highlights

- **2 High Severity Vulnerabilities:**

1. **[H-01] Hardcoded 18-Decimal Scaling:** Renders 8-decimal WBTC deposits virtually worthless (\$0.000006 per WBTC), prevents DSC minting, and causes liquidation transfers to fail due to attempting to seize 10^{18} WBTC tokens.
2. **[H-02] Broken Liquidation Health Factor Assertion:** Mathematically freezes liquidations for all positions where Health Factor ≤ 0.55 . As liquidators attempt to clear underwater debt, the 10% bonus reduces borrower health factors, triggering `DSCEngine__HealthFactorNotImproved` and accumulating unliquidatable bad debt.

- **4 Medium Severity Vulnerabilities:**

- [M-01] **Missing Non-Zero Oracle Price Validation:** Triggers division by zero in liquidation pricing.
- [M-02] **CEI Violation & Missing Reentrancy Protection:** Collateral is paid out before burning debt or checking health factor invariants.
- [M-03] **Permanent DSC Ownership Lock:** Full ownership of the DSC token is transferred to `dsc_engine` with zero governance or recovery hooks.
- [M-04] **Missing ZKsync L2 Sequencer Feed & 72-Hour Timeout:** Stale oracle prices expose the protocol to arbitrary arbitrage and bad debt.

- **2 Low / Informational Findings:**

- [L-01] **Duplicate Collateral Double Counting:** Lack of duplicate token checks in constructor allows 2x collateral inflation.
- [L-02] **Event Parameter Omission:** `CollateralDeposited` omits token address parameter.

2. Audit Scope & Metrics

Contract File	Language	Compiler	Purpose	SLoC
<code>src/dsc_engine.vy</code>	Vyper	~0.4.0	Core protocol vault, collateral accounting, mint/burn & liquidation	310
<code>src/oracle_lib.vy</code>	Vyper	~0.4.0	Chainlink price feed aggregator library & staleness circuit breaker	45

Contract File	Language	Compiler	Purpose	SLoC
<code>src/ decentralized_stable_coin .vy</code>	Vyper	~0.4.0	ERC20 stablecoin implementation (Snekmate extension)	35
Total In-Scope SLoC				390

3. Findings Summary & CodeHawks Metadata Table

Finding ID	Submission Title	Impact	Likelihood	Severity	Affected Scope	Status
[H-01]	Hardcoded 18-Decimal Scaling Causes 10-Order-of-Magnitude WBTC Collateral Undervaluation & Broken Liquidation Seizures	High	High	High	<code>src/ dsc_engine .vy</code>	Verified by PoC

Finding ID	Submission Title	Impact	Likelihood	Severity	Affected Scope	Status
[H-02]	Flawed Health Factor Mono-tonicity Assertion Blocks Liquidation of Underwater Accounts (HF $\leq$ 0.55), Causing Bad Debt Insolvency	High	High	High	src/dsc_engine.vy	Verified by PoC
[M-01]	Missing Zero and Negative Price Validation in oracle_lib.vy Triggers Division by Zero and Protocol Denial of Service	Medium	Medium	Medium	src/oracle_lib.vy	Verified by PoC

Finding ID	Submission Title	Impact	Likelihood	Severity	Affected Scope	Status
[M-02]	Checks-Effects-Interactions (CEI) Violation and Missing Reentrancy Guard in liqui-date()	Medium	Medium	Medium	src/dsc_engine.vy	Verified by Code Review
[M-03]	Permanent DSC Token Ownership and Minter Role Administrative Lock	High	Low	Medium	src/dsc_engine.vy	Verified by PoC
[M-04]	Missing ZKsync Era L2 Sequencer Uptime Feed Check & Hard-coded 72-Hour Staleness Timeout	Medium	Medium	Medium	src/oracle_lib.vy	Verified by Analysis

Finding ID	Submission Title	Impact	Likelihood	Severity	Affected Scope	Status
[L-01]	Duplicate Collateral Token Addresses Cause Collateral Double-Counting	High	Low	Low	src/dsc_engine.vy	Verified by PoC
[L-02]	CollateralDepositEvent Omits Collateral Token Address Parameter	Low	High	Low	src/dsc_engine.vy	Verified by Code Review
[Info]	Unused External Functions, Positional Event Syntax Deprecation, and Storage Inefficiencies	Low	Low	Info	Entire Scope	Verified by Compiler Log

4. The Tincho Method Hard-Gate Execution

Hard-Gate 0: Architecture & Protocol Flow Verification

1. Core Mint & Liquidation Sequence Flow

```

1 sequenceDiagram
2     autonumber
3     actor User as Borrower
4     actor Liquidator
5     participant DSCEngine as dsc_engine.vy
6     participant Collateral as Collateral Token (WETH / WBTC)
7     participant DSC as decentralized_stable_coin.vy
8     participant Oracle as Chainlink / oracle_lib.vy
9
10    User->>DSCEngine: deposit_collateral(token, amount)
11    DSCEngine->>Collateral: transferFrom(user, DSCEngine, amount)
12    User->>DSCEngine: mint_dsc(amount_to_mint)
13    DSCEngine->>DSCEngine: Check Health Factor >= 1.0
14    DSCEngine->>DSC: mint(user, amount_to_mint)
15
16    Note over User,Oracle: Market drops sharply -> User HF drops to
17    0.50
18    Liquidator->>DSCEngine: liquidate(collateral, user,
19    debt_to_cover)
20    DSCEngine->>Oracle: Fetch spot price
21    DSCEngine->>Collateral: transfer(liquidator, collateral + 10%
22    bonus)
23    DSCEngine->>DSC: burn_from(liquidator, debt_to_cover)
24    DSCEngine->>DSCEngine: assert ending_HF > starting_HF
25    Note over DSCEngine: BUG H-02: ending_HF (0.4944) < starting_HF
26    (0.5000)!
27    DSCEngine-->>Liquidator: REVERT "
28    DSCEngine__HealthFactorNotImproved"
```

2. Physical Custody & Asset Flow

```

1 flowchart TD
2     User([User([Borrower])]) -- "1. Deposit WETH / WBTC --> Engine["
3     dsc_engine.vy Custody Vault"] --> Engine[Engine]
4     Engine -- "2. Mint DSC Token --> User" --> User
5     User -- "3. Burn DSC --> Engine" --> Engine
6     Engine -- "4. Release Collateral --> User" --> User
7     Liquidator([Liquidator]) -- "5. Repay Bad Debt (DSC) --> Engine" --> Engine
8     Engine -- "6. Seize Collateral + 10% Bonus --> Liquidator" --> Liquidator
9
10    subgraph Oracle Architecture
11        Engine --> OracleLib[oracle_lib.vy]
12        OracleLib --> Chainlink[Chainlink Feeds]
13    end
```

Hard-Gate 1: Checks-Effects-Interactions (CEI) & FREI-PI Deconstruction

Function	Checks (C)	Effects (E)	Interactions (I)	CEI / FREI-PI Verdict
<code>deposit_collateral</code>	Validates <code>amount > 0 & is_allowed_token</code>	Updates <code>s_balances[user][token]</code>	External call: <code>transferFrom</code>	[PASS] Compliant
<code>mint_dsc</code>	Validates <code>amount > 0</code>	Updates <code>s_dsc_minted[user]</code>	Revert check → External <code>mint</code> call	[PASS] Compliant
<code>redeem_collateral</code>	Validates <code>amount > 0</code>	Decrements <code>s_balances[from][token]</code>	External call: <code>transfer</code>	[WARN] Invariant checked AFTER transfer
<code>burn_dsc</code>	Validates <code>amount > 0</code>	Decrements <code>s_dsc_minted[from]</code>	External call: <code>burn_from</code>	[PASS] Compliant
<code>liquidate</code>	Validates <code>debt > 0 & starting_hf < 1.0</code>	Decrements balance & debt	External call: <code>transfer</code> BEFORE <code>burn_from</code>	[FAIL] Violates CEI (M-02)

Hard-Gate 2: Executable PoC Suite (tests/unit/test_poc_audit.py)

All vulnerabilities identified in this report are accompanied by automated tests in `tests/unit/test_poc_audit.py`. Execution command:

```
1 mox test tests/unit/test_poc_audit.py -v
```

Results: - `test_poc_wbtc_8decimals_collateral_distortion`: **PASSED** (Confirms 10-order of magnitude WBTC valuation error). - `test_poc_liquidation_fails_when_hf_below_55`: **PASSED** (Confirms liquidation reverts with `DSCEngine__HealthFactorNotImproved` when $HF \leq 0.55$). - `test_poc_oracle_zero_price_causes_failure`: **PASSED** (Confirms unhandled division by zero when price is 0). - `test_poc_permanent_dsc_ownership_lock`: **PASSED** (Confirms DSC contract cannot be managed post-deployment). - `test_poc_duplicate_collateral_tokens_does_not_cause_failure`: **PASSED** (Confirms 2x artificial collateral inflation with duplicate tokens).

Hard-Gate 3: Invariant Mining & Stateful Fuzzing (tests/fuzz/test_invariant_audit.py)

1. Solvency Invariant (test_invariant_solvency_holds_for_valid_actions):

- Property: $\sum \text{Collateral USD Value} \geq \sum \text{DSC Total Supply}$.
- Verified across randomized deposit/mint ratios below threshold: **PASSED**.

2. Liquidation Invariant Breach (test_invariant_liquidation_cannot_clear_bad_debt_when_hf_dr):

- Property: For any underwater account ($HF < 1.0$), a solvent liquidator must be capable of reducing protocol debt.
 - Result: **BREACH DETECTED / PROVEN**. When prices fall such that $HF \leq 0.55$, liquidations fail 100% of the time.
-

Hard-Gate 4: Automated Mutation Testing Scorecard (scripts/mutation_runner.py)

A mutation test was conducted across 12 logic mutations within `src/dsc_engine.vy`.

- **Total Mutants Tested:** 12
- **Killed Mutants:** 7
- **Surviving Mutants:** 5
- **Empirical Mutation Score:** 58.3% (Target: $\geq 85\%$)

Blind Spot Analysis: The 5 surviving mutants exposed critical blind spots in the original test suite: - **MUT-01:** Boundary operator mutation in `_revert_if_health_factor_is_broken` ($\geq$ to $>$) was untested. - **MUT-05 & MUT-06:** Missing zero-value inputs for `_mint_dsc` and `liquidate`. - **MUT-07 & MUT-08:** Loose equality assertions on starting and ending health factor boundaries.

5. Detailed Findings & Recommendations

[H-01] Hardcoded 18-Decimal Scaling Causes 10-Order-of-Magnitude WBTC Collateral Undervaluation & Broken Liquidation Seizures

CodeHawks Submission Form Details

- **Title:** `Hardcoded 18-Decimal Scaling Causes 10-Order-of-Magnitude WBTC Collateral Undervaluation & Broken Liquidation Seizures`
- **Impact:** `High`
- **Likelihood:** `High`
- **Scope:** `src/dsc_engine.vy`

Vulnerability Details In `src/dsc_engine.vy`, the pricing functions hardcode 18-decimal precision assumptions:

```
1 PRECISION: constant(uint256) = 10**18
2 ADDITIONAL_FEED_PRECISION: constant(uint256) = 10**10
3 FEED_PRECISION: constant(uint256) = 10**8
```

In `_get_usd_value`:

```
1 @internal
2 @view
3 def _get_usd_value(token: address, amount: uint256) -> uint256:
4     price_feed: AggregatorV3Interface = AggregatorV3Interface(
5         self.s_price_feeds[token]
6     )
7     price: int256 = price_feed.stale_check_latest_round_data()
8     return (
9         (convert(price, uint256) * ADDITIONAL_FEED_PRECISION) *
10         amount
11     ) // PRECISION
```

And in `_get_token_amount_from_usd`:

```
1 @internal
2 @view
3 def _get_token_amount_from_usd(token: address, usd_amount_in_wei:
4     uint256) -> uint256:
5     price_feed: AggregatorV3Interface = AggregatorV3Interface(self.
6         s_price_feeds[token])
7     price: int256 = price_feed.stale_check_latest_round_data()
8     return (usd_amount_in_wei * PRECISION) // (
9         (convert(price, uint256)) * ADDITIONAL_FEED_PRECISION
10     )
```

On Ethereum and ZKsync Era, **WBTC uses 8 decimals** (1 WBTC = 10^8 satoshis). When a user deposits 1 WBTC (10^8 raw units) at a market price of \$60,000 (60000×10^8 from

Chainlink):

$$\text{Calculated USD} = \frac{(60000 \times 10^8 \times 10^{10}) \times 10^8}{10^{18}} = 60000 \times 10^8 \text{ wei} = \$0.000006 \text{ USD}$$

The actual collateral value is \$60,000 (60000×10^{18} wei). The protocol undervalues the collateral by a factor of 10^{10} (10 billion times). The user cannot even mint 1 DSC.

Conversely, when liquidating \$60,000 of debt against WBTC:

$$\text{Token Amount} = \frac{60000 \times 10^{18} \times 10^{18}}{60000 \times 10^8 \times 10^{10}} = 10^{18} \text{ tokens}$$

The contract calculates that the liquidator should receive 10^{18} WBTC raw units (10,000,000,000 WBTC), far exceeding the total supply of Bitcoin. The liquidation transaction reverts with an ERC20 transfer error, preventing liquidation of any WBTC position.

Impact Complete loss of functionality for WBTC collateral. Users who deposit WBTC cannot borrow against their collateral. Any position backed by WBTC can never be liquidated, leading to trapped assets and severe protocol insolvency.

Proof of Concept Verified in `tests/unit/test_poc_audit.py::test_poc_wbtc_8decimals_collateral_di`.

Recommended Mitigation Query the ERC20 token decimals dynamically or pass decimal scaling factors into the constructor:

```
1 +interface IERC20Metadata:
2 +    def decimals() -> uint8: view
3
4 @internal
5 @view
6 def _get_usd_value(token: address, amount: uint256) -> uint256:
7     price_feed: AggregatorV3Interface = AggregatorV3Interface(self.
8         s_price_feeds[token])
9     price: int256 = price_feed.stale_check_latest_round_data()
10 +    token_decimals: uint256 = convert(staticcall IERC20Metadata(
11         token).decimals(), uint256)
12 +    return (convert(price, uint256) * 10**10 * amount) // (10**
13         token_decimals)
```

[H-02] Flawed Health Factor Monotonicity Assertion Blocks Liquidation of Underwater Accounts ($HF \leq 0.55$), Causing Bad Debt Insolvency

CodeHawks Submission Form Details

- **Title:** Flawed Health Factor Monotonicity Assertion Blocks Liquidation of Underwater Accounts ($HF \leq 0.55$), Causing Bad Debt Insolvency
- **Impact:** High
- **Likelihood:** High
- **Scope:** `src/dsc_engine.vy`

Vulnerability Details In `src/dsc_engine.vy`, `liquidate()` enforces:

```
1  assert (
2      ending_user_health_factor > starting_user_health_factor
3  ), "DSCEngine__HealthFactorNotImproved"
```

The liquidator repays debt ΔD and seizes collateral of equal USD value plus a 10% liquidation bonus ($1.1\Delta D$). Let: - C = Collateral value in USD - D = Debt in DSC - Threshold $T = 0.50$ - Bonus $B = 1.10$

The initial health factor is:

$$HF_{start} = \frac{0.5 \cdot C}{D}$$

After liquidation:

$$HF_{end} = \frac{0.5 \cdot (C - 1.1\Delta D)}{D - \Delta D}$$

For $HF_{end} > HF_{start}$:

$$\frac{C - 1.1\Delta D}{D - \Delta D} > \frac{C}{D} \implies C\Delta D > 1.1D\Delta D \implies \frac{C}{D} > 1.10$$

Since $HF_{start} = 0.5\frac{C}{D}$, we have $\frac{C}{D} = 2 \cdot HF_{start}$. Substituting:

$$2 \cdot HF_{start} > 1.10 \iff HF_{start} > 0.55$$

Mathematical Proof of Failure: When a borrower's health factor drops to $HF \leq 0.55$, seizing a 10% bonus reduces the borrower's collateral faster than it reduces debt, causing $HF_{end} \leq HF_{start}$. The assertion strictly reverts with `"DSCEngine__HealthFactorNotImproved"`.

Impact Critical protocol insolvency. In any sudden market downturn where an account's health factor drops below 0.55, liquidations become mathematically impossible. The protocol cannot clear bad debt, causing the DSC stablecoin to lose its peg and collapse.

Proof of Concept Verified in `tests/unit/test_poc_audit.py::test_poc_liquidation_fails_when_hf_be` and invariant fuzzer `tests/fuzz/test_invariant_audit.py`.

Recommended Mitigation Do not enforce ending health factor monotonicity when an account is severely underwater, or allow full liquidations that completely clear the borrower's debt to 0:

```
1     ending_user_health_factor: uint256 = self._health_factor(user)
2 -     assert (
3 -         ending_user_health_factor > starting_user_health_factor
4 -     ), "DSCEngine__HealthFactorNotImproved"
5 +     if self.s_dsc_minted[user] > 0:
6 +         assert (
7 +             ending_user_health_factor > starting_user_health_factor
8 +             or starting_user_health_factor <= 55 * 10**16
9 +             ), "DSCEngine__HealthFactorNotImproved"
```

Alternatively, adopt standard lending protocol designs (such as Aave / MakerDAO) where the liquidator is permitted to liquidate up to 50% or 100% of debt without enforcing health factor monotonicity when the position is already below threshold.

[M-01] Missing Zero and Negative Price Validation in `oracle_lib.vy` Triggers Division by Zero and Protocol Denial of Service

CodeHawks Submission Form Details

- **Title:** Missing Zero and Negative Price Validation in `oracle_lib.vy` Triggers Division by Zero and Protocol Denial of Service
- **Impact:** Medium
- **Likelihood:** Medium
- **Scope:** `src/oracle_lib.vy`

Vulnerability Details In `src/oracle_lib.vy`:

```
1 @internal
2 @view
3 def _stale_check_latest_round_data(
4     chainlink_feed: AggregatorV3Interface,
5 ) -> int256:
6     round_id: uint80 = 0
7     price: int256 = 0
8     started_at: uint256 = 0
9     updated_at: uint256 = 0
```

```

10     answered_in_round: uint80 = 0
11     (
12         round_id,
13         price,
14         started_at,
15         updated_at,
16         answered_in_round,
17     ) = staticcall chainlink_feed.latestRoundData()
18
19     if updated_at == 0 or block.timestamp - updated_at > TIMEOUT:
20         raise "OracleLib__StalePrice"
21
22     return price

```

`price` is defined as `int256`. The library checks timestamp staleness but omits `price > 0`. If Chainlink returns 0 or negative price, `dsc_engine._get_token_amount_from_usd` executes:

```

1 return (usd_amount_in_wei * PRECISION) // (
2     (convert(price, uint256)) * ADDITIONAL_FEED_PRECISION
3 )

```

When `price == 0`, this causes an immediate division by zero revert. When `price < 0`, casting via `convert(price, uint256)` wraps around into an astronomical number in Vyper or causes an unexpected arithmetic error.

Impact Denial of service for liquidations and collateral valuation during abnormal market conditions or oracle hiccups.

Proof of Concept Verified in `tests/unit/test_poc_audit.py::test_poc_oracle_zero_price_causes_fail`.

Recommended Mitigation

```

1     if updated_at == 0 or block.timestamp - updated_at > TIMEOUT:
2         raise "OracleLib__StalePrice"
3 +     if price <= 0:
4 +         raise "OracleLib__InvalidPrice"

```

[M-02] Checks-Effects-Interactions (CEI) Violation and Missing Reentrancy Guard in `liquidate()`

CodeHawks Submission Form Details

- **Title:** Checks-Effects-Interactions (CEI) Violation and Missing Reentrancy Guard in `liquidate()`
- **Impact:** Medium
- **Likelihood:** Medium
- **Scope:** `src/dsc_engine.vy`

Vulnerability Details In `src/dsc_engine.vy`, `liquidate()` redeems collateral before burning the liquidator's DSC tokens:

```

1 # Interactions: External call transferring collateral token to
  liquidator
2 self._redeem_collateral(
3     token_collateral_address, token_amount_from_debt_covered, user,
      msg.sender
4 )
5
6 # Effects: Burning DSC debt
7 self._burn_dsc(debt_to_cover, user, msg.sender)
8
9 # Invariant check
10 ending_user_health_factor: uint256 = self._health_factor(user)
11 assert ending_user_health_factor > starting_user_health_factor, ...

```

Furthermore, `dsc_engine.vy` does not use `@nonreentrant` on any of its external functions.

Impact If collateral tokens support callbacks (e.g. ERC777 tokens, tokens with transfer hooks, or cross-chain bridged tokens), a malicious liquidator can reenter `dsc_engine` during `_redeem_collateral` before their DSC debt is burned or health factor invariants are validated.

Recommended Mitigation

1. Reorder execution to conform strictly to Checks-Effects-Interactions: burn DSC first, then transfer collateral.
2. Apply `@nonreentrant` to `liquidate()` and all state-changing vault entrypoints:

```

1 - self._redeem_collateral(token_collateral_address,
  token_amount_from_debt_covered, user, msg.sender)
2 - self._burn_dsc(debt_to_cover, user, msg.sender)
3 + self._burn_dsc(debt_to_cover, user, msg.sender)
4 + self._redeem_collateral(token_collateral_address,
  token_amount_from_debt_covered, user, msg.sender)

```

[M-03] Permanent DSC Token Ownership and Minter Role Administrative Lock

CodeHawks Submission Form Details

- **Title:** Permanent DSC Token Ownership and Minter Role Administrative Lock
- **Impact:** High
- **Likelihood:** Low
- **Scope:** `src/dsc_engine.vy`

Vulnerability Details In deployment script `script/deploy_dsc.py`:

```
1 dsc.set_minter(dsce.address, True)
2 dsc.transfer_ownership(dsce.address)
```

Ownership of `decentralized_stable_coin.vy` is transferred to `dsc_engine.vy`. However, `dsc_engine.vy` contains no functions to invoke `set_minter` or `transfer_ownership`.

Impact The DSC token's administrative control is permanently locked inside the engine. The protocol can never:

- Authorize a replacement engine (v2 migration).
- Add a Peg Stability Module (PSM) or flash minter.
- Transfer ownership to a DAO or multi-sig governance contract.

Proof of Concept Verified in `tests/unit/test_poc_audit.py::test_poc_permanent_dsc_ownership_lock`.

Recommended Mitigation Retain ownership of `decentralized_stable_coin` in a Timelock / Multi-sig contract, or add administrative passthrough functions to `dsc_engine.vy`.

[M-04] Missing ZKsync Era L2 Sequencer Uptime Feed Check & Hardcoded 72-Hour Staleness Timeout

CodeHawks Submission Form Details

- **Title:** Missing ZKsync Era L2 Sequencer Uptime Feed Check & Hardcoded 72-Hour Staleness Timeout
- **Impact:** Medium
- **Likelihood:** Medium
- **Scope:** `src/oracle_lib.vy`

Vulnerability Details The protocol targets deployment on **ZKsync Era**. When an L2 sequencer encounters downtime, transactions are paused while Layer 1 feeds may continue moving. Upon sequencer restart, stale prices are consumed if transactions execute before Chainlink feeds update on L2. Additionally, `TIMEOUT: constant(uint256)= 3 * 3600 * 24` (72 hours) is excessive; standard Chainlink feeds on L2 have heartbeat intervals between 1 hour and 24 hours.

Impact Arbitrageurs can exploit stale prices during sequencer reboots, unfairly liquidating healthy positions or borrowing underpriced DSC against overvalued collateral.

Recommended Mitigation

1. Integrate the Chainlink L2 Sequencer Uptime Feed on ZKsync Era with an enforced grace period (e.g. 3600 seconds).
2. Reduce the price feed staleness timeout to match asset heartbeat parameters (e.g. 3 hours for ETH/USD).

[L-01] Duplicate Collateral Token Addresses Cause Collateral Double-Counting

CodeHawks Submission Form Details

- **Title:** Duplicate Collateral Token Addresses Cause Collateral Double-Counting
- **Impact:** High
- **Likelihood:** Low
- **Scope:** `src/dsc_engine.vy`

Vulnerability Details In `dsc_engine.vy` constructor:

```
1 for i: uint256 in range(2):
2     price_feed: address = price_feed_addresses[i]
3     token: address = token_addresses[i]
4     self.s_price_feeds[token] = price_feed
5     self.s_collateral_tokens[i] = token
```

No validation prevents passing the same token address twice. When calculating total collateral value:

```
1 for token: address in self.s_collateral_tokens:
2     total_collateral_value_in_usd += self._get_usd_value(
```

```
3         token, self.s_balances[user][token]
4     )
```

If `token_addresses = [WETH, WETH]`, the loop processes `s_balances[user][WETH]` twice, artificially doubling the user's borrowing power.

Proof of Concept Verified in `tests/unit/test_poc_audit.py::test_poc_duplicate_collateral_tokens_`.

Recommended Mitigation Ensure all collateral token addresses in the constructor are unique:

```
1     assert token_addresses[0] != token_addresses[1], "
        DSCEngine__DuplicateCollateralToken"
```

[L-02] CollateralDeposited Event Omits Collateral Token Address Parameter

CodeHawks Submission Form Details

- **Title:** `CollateralDeposited Event Omits Collateral Token Address Parameter`
- **Impact:** `Low`
- **Likelihood:** `High`
- **Scope:** `src/dsc_engine.vy`

Vulnerability Details `CollateralDeposited` is defined as:

```
1 event CollateralDeposited:
2     user: indexed(address)
3     amount: indexed(uint256)
```

Whereas `CollateralRedeemed` is:

```
1 event CollateralRedeemed:
2     token: indexed(address)
3     amount_collateral: uint256
4     _from: indexed(address)
5     _to: indexed(address)
```

Indexers cannot determine which collateral asset was deposited without tracing transaction traces or reading storage.

Recommended Mitigation

```

1  event CollateralDeposited:
2      user: indexed(address)
3  +   token: indexed(address)
4      amount: indexed(uint256)

```

6. Automated Mutation Scorecard & Invariant Integrity

Testing is only as strong as its ability to detect flaws. The automated mutation runner injected 12 critical mutations into `dsc_engine.vy`:

```

1  +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2  | Mutation Testing Summary: 7 Killed, 5 Survived (58.3% Mutation
   | Score) |
3  +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

The survivors highlighted that previous testing lacked assertions on boundary values for `MIN_HEALTH_FACTOR` and zero-value debt coverage. Hardened invariant and unit tests have now been added to close these test gaps.

7. Conclusion

Algo Sstablecoinss features a clean architectural foundation, but contains two critical mathematical and precision flaws (H-01 and H-02) that prevent WBTC integration and cause total liquidation freezing during market volatility. Remediating these two High-severity findings along with the Medium-severity oracle and access-control issues will ensure the protocol remains solvent, robust, and production-ready on ZKsync Era.